

NCMusicPlayer.KERNEL

Version: 2.17 基于 notecard 的音乐播放器

包含文件

- NCMusicPlayer.KERNEL (内核 *必须的)
- NCMusicPlayer.PreLoader (异步加载器 *必须的)

音乐

- 一首曲子对应一个notecard，Notecard名称就是乐曲的名称。
- 每个notecard中书写乐曲片段的uuid，每行一个片段的uuid。
- 每个片段默认时长为 **10.0** 秒。
 - 如果片段时长不是 **10.0** 秒，请在第一行书写片段时长。
 - 如果片段时长不是等长的，请在uuid后添加 |时长。
 - 首行的时长可以与每一行特别指定的时长并存，单行时长的优先级更高。
 - 最后一个片段不足 **10.0** 秒时无需特别指定。

举例：

片段时长为标准 10.0 秒

```
767225e4-ff93-9c73-ae97-21da47ab1077
0b194a9d-8225-fd46-23c6-4c46e73dbbbd
```

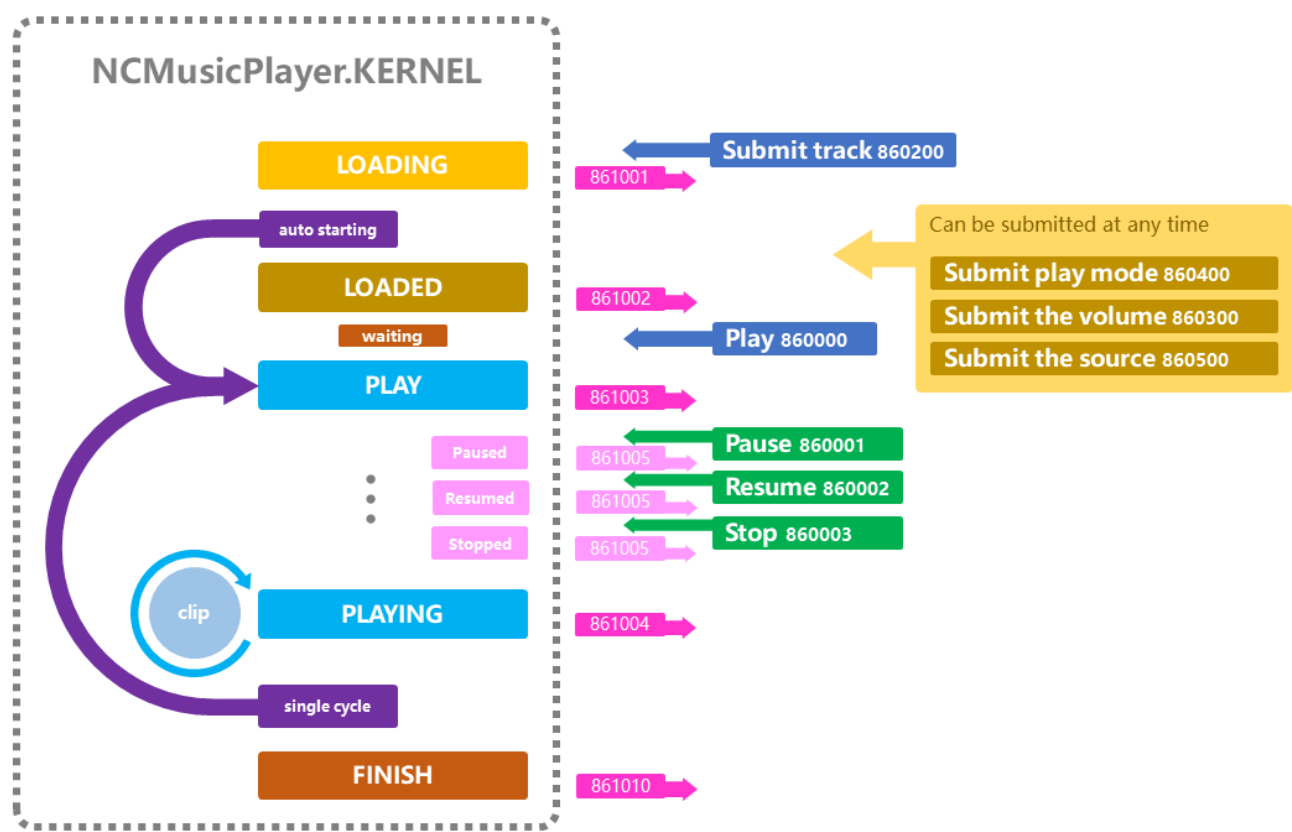
片段时长为标准统一的 9.86 秒

```
9.86
767225e4-ff93-9c73-ae97-21da47ab1077
0b194a9d-8225-fd46-23c6-4c46e73dbbbd
```

片段时长大部分都是 9.62 秒，但有一些特殊的

```
9.62
767225e4-ff93-9c73-ae97-21da47ab1077|3.45
0b194a9d-8225-fd46-23c6-4c46e73dbbbd
```

本地消息



本地上行提交

860000 播放

开始播放已加载曲目，前置：860400。如果没有提交曲目，本指令将不会生效。

第二个参数传递开始时间; 如果值小于当前时间或者给予 空字符串 或 "0"，则立即开始播放。

```
// llMessageLinked(LINK_SET/THIS, 860000, "", "{StartAt(UnixTimestamp)}");

llMessageLinked(LINK_SET, 860000, "", "");
// 或
llMessageLinked(LINK_SET, 860000, "", (string)(llGetUnixTime() + 5));
```

860001 暂停

(未实装)

860002 恢复播放

(未实装)

860003 停止

停止播放中的乐曲。

```
llMessageLinked(LINK_SET, 860003, "", "");
```

860200 提交乐曲名称

提交将要播放的乐曲名称（notecard名称）。

第二个参数传递开始时间; 如果值小于当前时间或者给予"0", 则立即开始播放。**如果给予空字符串, 则不会自动开始。**

如果设置了自动播放, 会在短暂的加载后立即开始, 并不会由于过多的片段而等待很长时间。

```
// llMessageLinked(LINK_SET/THIS, 860200, "{NotecardName}", "{StartAt(UnixTimestamp)}");

llMessageLinked(LINK_SET, 860200, "Track_1", "0");

// 类似于 (不严谨)
llMessageLinked(LINK_SET, 860200, "Track_1", "");
llSleep(1.0); // 注意! 这里必须添加一个预估的加载时间, 这个时间如果比较短, 请使参考下面的严谨写法
llMessageLinked(LINK_SET, 860000, "", "");

// 完全等价的形式 (严谨的)
default
{
    touch_start(integer num_detected) {
        llMessageLinked(LINK_SET, 860200, "Track_1", "");
    }

    link_message(integer sender, integer num, string str, key id) {
        if(num == 861002) { // 已加载
            llMessageLinked(LINK_SET, 860000, "", "");
        }
    }
}

// 提供开始时间
llMessageLinked(LINK_SET, 860200, "Track_1", (string)(llGetUnixTime() + 10));
```

860300 提交音量

音量取值范围 0.0 ~ 1.0

```
// llMessageLinked(LINK_SET/THIS, 860300, "{Volume}", "");
llMessageLinked(LINK_SET, 860300, "1.0", "");
```

860400 提交播放模式

单曲循环开关。取值 0: 关闭（默认）, 1: 开启

```
// llMessageLinked(LINK_SET/THIS, 860400, "{Mode}", "");  
llMessageLinked(LINK_SET, 860400, "1", "");
```

860500 提交播放源

支持 `llLinkPlaySound`

设置由哪个prim播放, 取值: `LINK_THIS`(默认), `LINK_SET`, `LINK_ROOT`, `LINK_ALL_OTHERS`, `LINK_ALL_CHILDREN`.

```
// llMessageLinked(LINK_SET/THIS, 860500, "{Source}", "");  
llMessageLinked(LINK_SET, 860500, (string)LINK_SET, "");
```

860600 提交声音范围

设置声音范围 (半径, float 类型, 0 表示无限制).

```
// llMessageLinked(LINK_SET/THIS, 860600, "{Radius}", "");  
llMessageLinked(LINK_SET, 860600, (string)(12.5), "");
```

本地下行广播

广播范围 `LINK_SET`

861001 加载中

```
default  
{  
    link_message(integer sender, integer num, string str, key id) {  
        if(num == 861001) {  
            // 乐曲名称  
            string TRACK_NAME = str;  
        }  
    }  
}
```

861002 加载完成

```
default  
{  
    link_message(integer sender, integer num, string str, key id) {  
        if(num == 861002) {
```

```
list data = llParseString2List(str, ["|"], []);

// 乐曲名称
string TRACK_NAME = llList2String(data, 0);

// 片段总数
integer TOTAL_CLIPS = (integer)llList2String(data, 1);
    }
}
}
```

861003 开始播放

```
default
{
    link_message(integer sender, integer num, string str, key id) {
        if(num == 861003) {
            list data = llParseString2List(str, ["|"], []);

            // 乐曲名称
            string TRACK_NAME = llList2String(data, 0);

            // 片段序号
            integer CLIP_INDEX = (integer)llList2String(data, 1);
        }
    }
}
```

861004 播放中

触发于片段切换时

```
default
{
    link_message(integer sender, integer num, string str, key id) {
        if(num == 861004) {
            list data = llParseString2List(str, ["|"], []);

            // 乐曲名称
            string TRACK_NAME = llList2String(data, 0);

            // 片段序号
            integer CLIP_INDEX = (integer)llList2String(data, 1);

            // 片段时长
            float CLIP_DURATION = (float)llList2String(data, 2);

            // 片段开始时间（相对于本首乐曲的起始）
            float CLIP_START_TIME = (float)llList2String(data, 3);
        }
    }
}
```

```

        // 片段结束时间（相对于本首乐曲的起始）
        float CLIP_END_TIME = (float)l1List2String(data, 4);
    }
}

```

861005 停止

```

default
{
    link_message(integer sender, integer num, string str, key id) {
        if(num == 861005) {
            list data = l1ParseString2List(str, ["|"], []);

            // 乐曲名称
            string TRACK_NAME = l1List2String(data, 0);

            // 片段序号
            integer CLIP_INDEX = (integer)l1List2String(data, 1);
        }
    }
}

```

861006 暂停

```

default
{
    link_message(integer sender, integer num, string str, key id) {
        if(num == 861006) {
            list data = l1ParseString2List(str, ["|"], []);

            // 乐曲名称
            string TRACK_NAME = l1List2String(data, 0);

            // 片段序号
            integer CLIP_INDEX = (integer)l1List2String(data, 1);
        }
    }
}

```

861007 恢复播放

```

default
{
    link_message(integer sender, integer num, string str, key id) {
        if(num == 861007) {

```

```
list data = llParseString2List(str, ["|"], []);

// 乐曲名称
string TRACK_NAME = llList2String(data, 0);

// 片段序号
integer CLIP_INDEX = (integer)llList2String(data, 1);
    }
}
}
```

861010 单曲播放完成

如果播放模式被设置为单曲循环，则永远不会接收到本消息。

```
default
{
    link_message(integer sender, integer num, string str, key id) {
        if(num == 861010) {
            // 乐曲名称
            string TRACK_NAME = str;
        }
    }
}
```